

Setting up Hadoop over Lustre

Machine IP	Name	Lustre component
192.168.1.185	MDS	Metadata Server
192.168.1.184	OSS1	Object Server Storage #1
192.168.1.186	OSS2	Object Server Storage #2
192.168.1.188	Client1	Lustre Client #1
192.168.1.189	Client2	Lustre Client #2
192.168.1.193	Client3	Lustre Client #3
192.168.1.183	Client4	Lustre Client #4

Lustre, on the other hand, is an open source distributed parallel file system. It is a scalable, secure, robust and highly-available cluster file system that addresses I/O needs such as low latency and the extreme performance of large computing clusters. Lustre is basically an object-based file system. It is composed of three functional components: metadata servers (MDSs), object storage servers (OSSs) and clients. An MDS provides metadata services. It stores file system metadata such as file names, directories and permissions. The availability of the MDS is critical for file system data. In a typical configuration, there are two MDSs configured for high-availability failover. Since an MDS stores only metadata, the storage or metadata target (MDT) attached to the file system need only store hundreds of gigabytes for a multi-terabyte file system. One MDS per file system manages one MDT. Each MDT stores file metadata, such as file names, directory structures and access permissions.

Why Lustre and not HDFS?

The Hadoop Distributed File System works well for general statistical applications, but it might exhibit performance bottlenecks for complex computational applications like HPC or high performance computing-based applications that generate large, even increasing outputs. Second, HDFS is not POSIX-compliant, which means it cannot be used as a normal file system, which makes it difficult to extend. Also, HDFS has a WORM (write-once, read-many) access model, so changing even a small part of a file requires that all file data be copied, resulting in very time-intensive file modifications. Hadoop implements a computational paradigm named MapReduce, where the Reduce node uses HTTP to shuffle all related big Map Task outputs before the real task begins. This consumes a massive amount of resources and generates a lot of I/O and merge spill operation.

Setting up the Metadata server

I assume that you have CentOS/RHEL 6.x installed on your hardware. I have RHEL 6.x available and will be using it for this demonstration. This should work for CentOS 6.x

```
[root@MDS ~]# cat /etc/selinux/config
[root@MDS ~]# cat /etc/selinux/config
# This file controls the state of SELinux on the system.
# SELinux can take one of three values:
# enforcing - SELinux security policy is enforced.
# permissive - SELinux prints warnings instead of enforcing.
# disabled - No SELinux policy is loaded.
SELINUX=disabled
# SELinux can take one of two values:
# targeted - Targeted processes are protected,
# mls - Multi Level Security protection.
SELINUXTYPE=targeted
```

Figure 1: SELinux settings

```
[root@MDS ~]# uname -a
Linux MDS 2.6.32-279.14.1.el6_lustre.x86_64 #1 SMP Fri Dec 14 23:22:17 PST 2012
x86_64 x86_64 x86_64 GNU/Linux
[root@MDS ~]#
```

Figure 2: Kernel in use

```
[root@MDS ~]# pvcreate /dev/sda2
[root@MDS ~]# lvcreate --name vg00/mdt 6G
[root@MDS ~]# vgcreate vg00 /dev/sda2
[root@MDS ~]# lvcreate --name vg00/mdt --size 6G
[root@MDS ~]# ls -ls /dev/vg00
total 9
drwxr-xr-x. 2 root root 60 Mar 4 21:02 .
drwxr-xr-x. 15 root root 3940 Mar 4 21:02 ..
lrwxrwxrwx. 1 root root 7 Mar 4 21:02 mdt -> ../dm-0
```

Figure 3: Creating LVM

versions too. The firewall and SELinux both need to be disabled. You can disable the firewall using the `iptables` command, whereas SELinux can be disabled by changing it in the file `/etc/sysconfig/selinux`

SELINUX=disabled

Next, let's install Lustre-related packages through the Yum repository.

```
#yum install lustre
```

Reboot the machine to boot into the Lustre kernel. Once the machine is up, verify the Lustre kernel through the following command:

```
#uname -arn
Linux MDS 2.6.32-279.14.1.el6_lustre.x86_64 #1 SMP x86_64
x86_64 x86_64 GNU/Linux
```

Create a LVM partition on `/dev/sda2` (ensure this is a LVM partition type at your end). Refer to Figure 3.

```
#pvcreate /dev/sda2
#vgcreate vg00 /dev/sda2
#lvcreate -name vg00/mdt -size 6G
#ls -al /dev/vg00
```